

Moment Search: A Scalable *Multimodal* RAG Pipeline

Video, research papers and slide decks answering the same question, from one index — benchmarked under load, killed mid-ingest, and measured where it hurts.

Kurt Lozier · July 29, 2026 · corpus: 4 videos · 5 papers · 9 slide decks

180 ms

accept p95, n=200

12/14 (0.86)

recall@10, 14 queries

40/43 (0.93)

citation accuracy

14 / 14

recovered from kill -9

A video-search demo has to become a mixed-corpus system that survives a real backfill

Papers and decks hold the answers video does not. Splitting them into a second system splits the answer — so they enter the same queue and land in the same index, and we prove that under load nothing slows down and nothing is lost.

3.1

Multi-source ingestion

Parse, chunk, enrich and embed PDFs and decks into the one Qdrant index video already uses. Mixed sources become one searchable space.

3.2

Async work queues

A document flow on the existing Prefect queue, matching video's status lifecycle and retries. Accepting work is decoupled from doing it.

3.3

Scale & decoupling proof

Benchmark accept latency, throughput and recall. Keep search fast during a backfill. Kill a worker mid-ingest and prove nothing is lost.

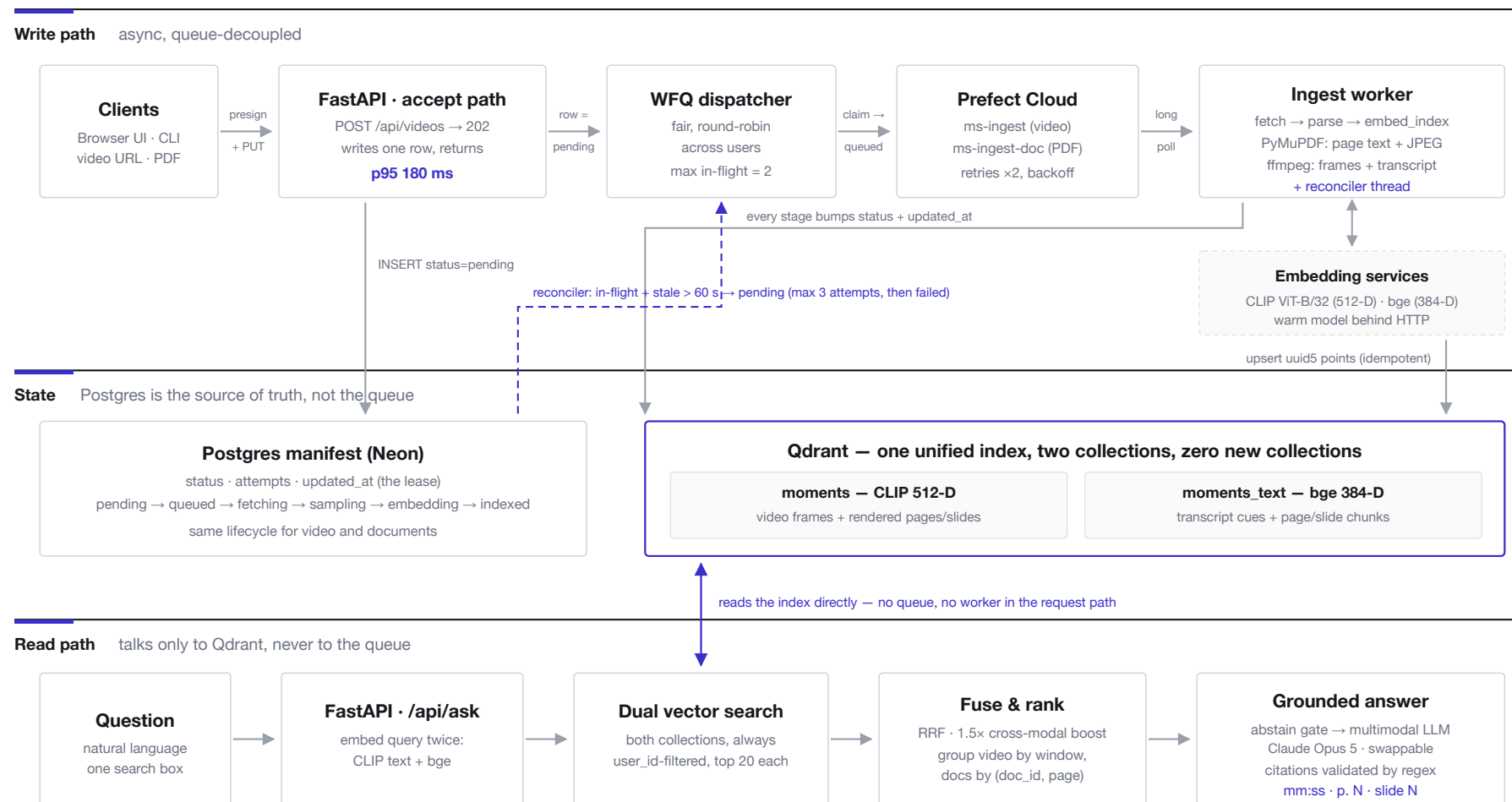
3.4

Think like an FDE

Managed vs self-hosted queues, citations that always resolve, workers that recover from a signal, and an honest path from demo to production.

The bar we set ourselves, because the brief set none: **accept p95 < 250 ms · search ≤ 1.25× under backfill · recall@10 ≥ 0.85 · citation accuracy ≥ 0.90 · zero loss on kill -9.**

One index, two paths — search reads Qdrant while the queue is still working



A slide is a picture — so we render every page through CLIP and make decks **visually searchable**

The obvious build indexes document text and stops. That treats a deck as a bad paper. Rendering each page into the same 512-D CLIP space video frames already occupy costs nothing extra at query time and buys a whole retrieval channel.

One searchable space, zero new collections

Documents reuse the collections video already had.

- Page and slide JPEGs → **moments** (CLIP 512-D)
- Page and slide text chunks → **moments_text** (bge 384-D)
- Query embedding is unchanged — same two branches, always both
- Multi-tenant `user_id` filter still holds

For a document, the page is the moment

Fusion needed one substitution, not a rewrite.

- Video hits group by time window; document hits by `(doc_id, page)`
- A page matching visually *and* textually earns the same **1.5× cross-modal boost** a video moment does
- Point ids are `uuid5(id:n)` — re-ingest overwrites, duplicates impossible by construction

In run **20260728B** all **140 citations across 14 answers carried a locator**: 83 timestamps, 42 p. N, 15 slide N. The LLM prompt, the UI card and the fallback answer never learned which corpus they were reading.

We refused native .pptx on purpose — and declare the bill we didn't pay

Rendering PowerPoint faithfully means LibreOffice headless inside the worker image. Every deck tool exports PDF in one click. The user cost is one menu item; the engineering cost is four separate problems.

What .pptx support would have cost

- **~500 MB+ image growth** on a build that today needs one wheel — PyMuPDF already does text *and* rendering
- **Cold-start conversion of seconds per file**, landing inside the window the throughput benchmark measures — 1.33 pages/s would have described LibreOffice's startup, not our pipeline
- **Silent fidelity loss** — missing fonts, SmartArt, embedded charts degrade without an error to show in the status field
- **A new OOM class** on a 2 GB worker already holding CLIP and bge, inside the component whose durability we are trying to prove

What the decision actually cost users

- **Nothing measurable.** 9 of the 14 corpus documents are decks; 7 were exported from .pptx — the filenames still say so
- All 7 ingested with **zero code beyond the PDF path**

Fast-follow — sketched, not built

- LibreOffice as a **sidecar** with its own image and memory limit, exposing `convert(pptx) → pdf`, called before parsing
- Keeps the worker image small, isolates the OOM, makes conversion latency separately measurable

A benchmark you cannot interpret is worse than one you never ran. **Naming the cut is scoping; building it today would have cost us the number.**

Documents enter through the video endpoint and land in the video index — no parallel system

One new branch on `POST /api/videos` keyed on content type, one new parser module, and a new id prefix. Everything downstream — thumbnails, citations, status chips, the fair dispatcher — was already generic enough to reuse.

01 · PARSE

PyMuPDF

One dependency covers both jobs: page-aware text extraction and page → JPEG rendering. No system packages beyond the wheel.

02 · CHUNK

Locator-aware

→ Papers: ~400-token page-aware chunks, 15% overlap, span {page_start, page_end}.
Decks: one chunk per slide.

03 · ENRICH

Payload & images

→ Every page rendered to JPEG on the existing frame key layout. Payload carries modality, kind and the locator.

04 · EMBED

Two spaces, locally

→ Pages through CLIP ViT-B/32 (512-D), chunks through bge (384-D). No LLM captioning step — both embedders run on CPU.

05 · INDEX

Idempotent upsert

→ Into the same two Qdrant collections as video, under uuid5 point ids. Re-ingest overwrites in place.

Proven at corpus scale: **237 pages indexed across 12 documents in one 178.5 s backfill**, plus 2 documents correctly recognised as already-indexed and marked skipped by content dedup — a correct outcome, counted as success.

A second Prefect deployment reused video's lifecycle verbatim — the UI needed zero changes

Parity was the requirement, so we spent it as a feature: the same status strings mean the status chips, the fair dispatcher and the progress bar work for documents without knowing documents exist.

The flow

- `ms-ingest-doc`: `fetch` → `parse` → `embed_index`, served alongside the video deployment in one worker process
- Identical lifecycle: `pending` → `queued` → `fetching` → `sampling` → `embedding` → `indexed`
- Retries mirror video's per-task pattern: `fetch` ×2 with backoff, `embed` ×2
- The WFQ dispatcher is source-agnostic — `enqueue` just picks a deployment by id prefix

Where the queue can't help

- **No drainable DLQ.** Prefect's dead-letter equivalent is a badge in a dashboard
- **A `kill -9` raises no exception** — the run is marked *Crashed* and nothing reschedules it
- **Poll latency** — a 3 s dispatch tick plus Prefect's long-poll is dead air inside the wall clock (it never touches accept latency)

So the queue is not the source of truth

- Every row carries `status`, `attempts`, `updated_at`; a reconciler thread returns stale in-flight rows to `pending`, or to `failed` past 3 attempts

Decoupling in one number: the `accept` call returns **202 in 180 ms p95** — and **197 ms p95 even while 14 documents were being ingested**. The uploader never waits on the worker.

Every target met — and the two numbers that **don't flatter us** are on the slide

Accept p95 · POST /api/videos
target < 250 ms

180 ms

n=200 · p50 173 ms **PASS**

Search p95 during backfill
target $\leq 1.25\times$ baseline

1.05 $\times$

LLM path · see next slide **PASS**

Recall@10 · 14 queries
target ≥ 0.85

12/14 (0.86)

video 1.00 · doc 1.00 **PASS**

Citation accuracy
target ≥ 0.90

40/43 (0.93)

40 ✓ / 3 ✗ claim-bearing **PASS**

Durability · SIGKILL mid-ingest
target: no loss, no dupes

14 / 14

0 lost · 0 failed · 218.9 s **PASS**

Citation locator coverage
target 100%

140 / 140

83 mm:ss · 42 p. N · 15 slide N **PASS**

Ingest throughput
report; stretch > 4.0/s

1.33 pages/s

237 pages / 178.5 s · one worker **REPORTED**

Cost per document
report

not measured

designed, never instrumented **OPEN**

Verified by **74 unit tests** and four benchmark scripts that talk to the running app over HTTP — nothing imports the app's internals, so every number here is end-to-end.

Answer generation: Claude Opus 5 (.env-configured, swappable). Citation accuracy, answer latency and the ask_full ratio are model-dependent; recall, accept latency, throughput and the kill test involve no LLM. Values rounded; count-based scores shown as fractions; exact figures in docs/bench/.

Sources: accept_200.json · backfill_20260728A.json · eval_20260728B.json · kill_worker_20260728_220849.json · citation_check_20260728B.md

Search survived the backfill — but the honest ratio is the one without the LLM in it

The SLO series passed at 1.05×. It passed partly because ~15 s of model latency swamps tens of milliseconds of contention. So we measured a second series with the LLM removed, and reported both.

Series	Baseline p95	During backfill p95	Ratio	What it tells you
ask_full — real question: retrieval + gate + multimodal LLM	15,178 ms	15,882 ms	1.05×	What a user actually waits. Passes the $\leq 1.25\times$ target — but the ratio is nearly insensitive by construction, and we say so.
ask_retr — same question, empty scope: both query embeddings and both Qdrant searches still run, zero tokens	882 ms	2,141 ms	2.43×	The retrieval path under contention. n=14, one during-backfill round. A proxy: it excludes HNSW traversal, fusion and citation assembly — use the ratio, not the value.
Accept path — POST /api/videos during the same backfill	180 ms	197 ms	flat	Registration is untouched by ingest load — the decoupling claim at its strongest. Not a like-for-like ratio: the during-backfill figure is the 14 real document registrations, not the n=200 standalone run.

The reading: the read path is decoupled in *architecture* — it never enters the queue — but on one laptop a query's CLIP embedding contends with a backfill's on the same CPU. That is a capacity finding, not a design failure, and it has a named fix: move CLIP off the query box.

kill -9 mid-embedding cost nothing, because recovery is state-driven, not exception-driven

A crashed worker does not raise, does not run a `finally`, never sets a status. Idempotency alone does not save you — the reference solution scored 0 of 4 on its first resilience run with perfectly correct idempotent writes, because nothing re-ran the work.

SETUP

14 PDFs registered

Queue draining normally, 2 documents in flight.

KILL

SIGKILL the worker

Both in-flight rows sitting at status: embedding. Held down 20 s.

SWEEP

Reconciler finds them

updated_at is the lease: in-flight and untouched > 60 s = orphaned. Interval 10 s, max 3 attempts.

RESULT

14/14 indexed

0 lost · 0 failed · 0 non-terminal · 218.9 s elapsed.

The proof is in attempts

- The **2 rows in flight at the kill: attempts: 2** — they re-ran, once each
- The other **12 rows: attempts: 1** — still pending, never claimed, never redone
- Exactly the work that was lost was repeated. Nothing else was.

Three things make it work

- **Idempotent writes** — `uuid5` point ids, so re-ingest overwrites; duplicates impossible by construction, not by a dedup pass
- **Liveness without a heartbeat service** — every state-changing call bumps `updated_at`
- **Something that re-triggers** — the reconciler is the only component that turns a dead row back into runnable work

The eval and the citation audit each found a **real bug** the unit tests could not

Both were caught the night before submission, both were fixed with tests, and both moved a headline number.

Bug 1 — one prolific source flooded the ranking

First full eval: recall **11/14 (0.79) — FAIL**. Video-only 5/5, doc-only 5/5, **mixed-corpus 1/4**. Excellent at single-corpus questions; bad at the one thing the system exists for.

- On one mixed query, 9 of 10 citations came from 4 papers — 5 from a single paper. On another, 10 of 10 were video.
- Fusion worked; the top-K did not.
- **Fix: MAX_PER_SOURCE = 3**, nine lines after sorting. Each source keeps its best three; the rest are **demoted, not dropped**.
- **Recall 11/14 → 12/14 (0.79 → 0.86) — PASS**; mixed 1/4 → 2/4; video and doc unchanged at 1.00.

Bug 2 — a citation pointing one page short

Every locator was scored against the sentence attached to it. A merely topical citation scores against you.

- Both wrong-locator findings had one root cause: **a chunk straddling a page boundary was cited to the page it starts on**, so a quote from its tail pointed one page early.
- The chunk always knew its span. The label now reads pp. 9–10 (commit 96445a5).
- **Citation accuracy: 40/43 (0.93)** — 40 ✓ / 3 ✗ of the claim-bearing citations.
- AI-assisted first pass, human spot-check pending — flagged in the file itself.

Two misses remain, **analysed rather than argued away**: the cap bounds one source but does not balance corpora, and a short true chunk with low lexical overlap loses to a vivid frame. Neither fix is one line.

Prefect won on ops burden, not capability — so we paid for it deliberately and said exactly where

MANAGED vs SELF-HOSTED

Every rejection was a cost argument

Redis: not built for multi-step workflows.
Custom queue: engineering overhead.
Temporal: costly and complex to operate.
None said Prefect retries better. It won because a team with no platform engineer can run it — and we accepted the poll latency and the missing DLQ that come with that.

GROUNDING CITATIONS

Locator or nothing, in three layers

An abstain gate on raw cosine before any LLM call (zero tokens on an unanswerable question); a regex that strips out-of-range [n] references from the answer text rather than asking the model to behave; and citation accuracy scored as a metric, not asserted.

RESILIENCE

Postgres is the source of truth

The queue holds work; the database holds state. status, attempts and updated_at make orphan detection a one-line predicate, and the reconciler's exhausted branch is a poor-man's DLQ — labelled as one, not dressed up.

DEMO → PRODUCTION

Four changes, one lens

N workers autoscaled on queue depth (test first for a model lock that serialises them); GPU CLIP off the query box, which fixes the 2.43× contention and throughput at once; shard and replicate Qdrant, keeping the tenant filter; tune the stuck window to the longest healthy gap.

The OPENS, stated plainly. **Cost per answer is now measured live** — the deployed /metrics page reports **~\$0.04 per answered question** on Opus 5 (first day). **Cost per document is still open:** ingest API spend is genuinely \$0 — no LLM captioning step, both embedders local on CPU — so the real cost is machine-seconds we still do not record. **Throughput is one worker's number.** Closing the rest needs a worker-seconds counter per document and a price table; then every trade-off above collapses into dollars per document ingested and dollars per question answered — the numbers a client actually signs. Model A/B (Kimi, GLM) is then a config swap plus an eval re-run.

A system whose numbers you can re-run — including the ones that aren't flattering

Deliverables

- **Deployed app on Fly.io** — live at <https://momentsearch-lozierk.fly.dev> (public corpus: four talks + five papers) with a live `/metrics` page; api, worker and clip process groups, one release command
- **Repo** with the document parser, the second Prefect flow, the reconciler and **122 tests** (the starter repo had none)
- **Benchmark harness** — `bench_accept.py`, `bench_backfill.py`, `kill_worker_test.sh`, `run_eval.py`
- **Golden set of 14 queries** — 5 video-only, 5 document-only, 4 mixed-corpus scored strictly: a mixed query counts only if a video *and* a document expectation both appear
- **Written FDE analysis** with every figure traced to a JSON in `docs/bench/`

What we'd do next, in order

- **Instrument cost per document** — worker-seconds per document and a price table, now that `/metrics` prices an answer. Two numbers replace every latency argument.
- **Move CLIP to its own box** and re-measure the 2.43× retrieval ratio with the query path off the ingest CPU.
- **Scale to N workers** and check for a model lock before believing any throughput multiple.
- **Per-corpus quota in fusion** — the fix the two remaining mixed-query misses point at.
- **Human spot-check** of the 40/43 (0.93) citation pass, starting with the 3 *X* rows.

The claim is not that it is fast. The claim is that **every number here was produced by a script anyone can run again** — and where the number was inconvenient, it is on the slide anyway.